Response Handling

Section: 8 — Request/Response Handling, Forms, and Validation

Subtopic: 02 — Response Handling

Estimated Duration: 2–2.5 hours

Theory Practical

Topics covered: `res.send` header behaviour, `res.json` edge cases and configuration, `res.status` comprehensive reference, `res.end`, `res.redirect`, setting headers with `res.set` and `res.type`, sending files with `res.sendFile` and `res.download`, content negotiation with `res.format`, response lifecycle rules

Prerequisites: Subtopic 07-01 (`res.send`, `res.json`, `res.status` basics)

Learning Objectives — This Subtopic

- Send responses in various formats using `res.send()`, `res.json()`, and `res.end()` and explain when to use each
- Set HTTP status codes correctly with `res.status()` and chain them with response methods
- Control response headers using `res.set()`, `res.type()`, and `res.append()`
- Redirect clients to different URLs with `res.redirect()` using appropriate status codes
- Serve files for display and download using `res.sendFile()` and `res.download()`
- Implement content negotiation with `res.format()` to serve different response types based on the client's `Accept` header

The Response Object

In subtopic 08-01, you explored the `req` object — how data arrives at the server. This subtopic covers the other half of the cycle: the `res` object and how to send data back to the client.

The `res` object in Express wraps Node's native `http.ServerResponse` (Section 6) and adds a large set of convenience methods. Where the raw HTTP module required you to manually call `res.writeHead()`, `res.write()`, and `res.end()`, Express provides methods like `res.json()` and `res.send()` that handle headers, serialisation, and encoding automatically.

Set up a project to follow along:

```
mkdir response-handling && cd response-handling
npm init -y
npm install express
```

```
response-handling/
├── node_modules/
├── public/
│   └── report.pdf
├── package.json
├── package-lock.json
└── app.js
```

```
mkdir public
```

> **Analogy:** If the `req` object is an incoming letter (the client's message to the server), the `res` object is the reply envelope. Express gives you a variety of pre-addressed, pre-stamped envelopes for different situations — `res.json()` for data, `res.redirect()` for forwarding, `res.sendFile()` for attachments — instead of making you hand-address every envelope yourself.

`res.send()` — The General-Purpose Sender

> **Cross-reference:** The basics of `res.send()` — auto-detecting Content-Type from argument type (string → HTML, object → JSON, Buffer → octet-stream) — were introduced in **subtopic 07-01 (Express Introduction), Section 5.1**. This section examines what happens at the HTTP header level.

Use the `-i` flag with `curl` to inspect the response headers that `res.send()` generates for each data type:

`app.js`

```
const express = require('express');
const app = express();

// Sending a string — Content-Type: text/html
app.get('/text', function (req, res) {
    res.send('<h1>Hello World</h1>');
});

// Sending a Buffer — Content-Type: application/octet-stream
app.get('/buffer', function (req, res) {
    res.send(Buffer.from('Raw binary data'));
});

// Sending an object or array — Content-Type: application/json
app.get('/object', function (req, res) {
    res.send({ name: 'Alice', role: 'developer' });
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
node app.js
```

Test each route and inspect the headers:

```
curl -i http://localhost:3000/text
```

Output:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 22

<h1>Hello World</h1>
```

```
curl -i http://localhost:3000/buffer
```

Output:

```
HTTP/1.1 200 OK
Content-Type: application/octet-stream
Content-Length: 15

Raw binary data
```

```
curl -i http://localhost:3000/object
```

Output:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 38

{"name":"Alice","role":"developer"}
```

The `-i` flag tells `curl` to display response headers alongside the body. Notice how Express automatically detects the data type and sets the appropriate `Content-Type`. It also calculates and sets `Content-Length`, and adds an `ETag` header for caching (omitted above for brevity).

Key Insight: When you pass an object or array to `res.send()`, Express internally calls `JSON.stringify()` and sets the content type to `application/json`. This means `res.send({ key: 'value' })` and `res.json({ key: 'value' })` produce identical output. The difference is intent and readability — covered next.

`res.json()` — Sending JSON Data

> **Cross-reference:** The basics of `res.json()` — serialising objects to JSON, setting `Content-Type: application/json`, and the difference from `res.send()` when passing objects — were introduced in **subtopic 07-01 (Express Introduction), Section 5.2**. This section covers edge cases and configuration options.

`res.json()` has two advantages over passing objects to `res.send()` that become important as your API grows:

1. **Clarity of intent.** When reading code, `res.json()` immediately signals that this route returns JSON. `res.send()` could be sending anything.
2. **It converts non-object types to JSON.** `res.json(null)`, `res.json(true)`, and `res.json(42)` all produce valid JSON responses, whereas `res.send(null)` sends an empty response and `res.send(42)` is interpreted as a status code (deprecated behaviour).

app.js

```
const express = require('express');
const app = express();

app.get('/api/user', function (req, res) {
    res.json({
        id: 1,
        name: 'Alice',
        active: true,
        tags: ['admin', 'developer']
    });
});

// JSON with special values
app.get('/api/null', function (req, res) {
    res.json(null);
});

app.get('/api/count', function (req, res) {
    res.json(42);
});
```

```
app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl http://localhost:3000/api/user
```

Output:

```
{"id":1,"name":"Alice","active":true,"tags":["admin","developer"]}
```

```
curl http://localhost:3000/api/null
```

Output:

```
null
```

```
curl http://localhost:3000/api/count
```

Output:

```
42
```

All three responses have `Content-Type: application/json`. The values `null` and `42` are valid JSON on their own — `res.json()` handles them correctly.

Pretty-Printing JSON

During development, you may want JSON responses to be human-readable. Express provides an application setting for this:

app.js (with pretty JSON)

```
const express = require('express');
const app = express();

// Enable pretty-printed JSON with 2-space indentation
app.set('json spaces', 2);

app.get('/api/user', function (req, res) {
```

```
    res.json({ id: 1, name: 'Alice', role: 'developer' });
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl http://localhost:3000/api/user
```

Output:

```
{
  "id": 1,
  "name": "Alice",
  "role": "developer"
}
```

`app.set('json spaces', 2)` tells Express to pass the `2` argument to `JSON.stringify(data, null, 2)`. This increases response size, so disable it in production — or make it conditional:

`app.js (conditional pretty-printing)`

```
if (process.env.NODE_ENV !== 'production') {
    app.set('json spaces', 2);
}
```

`res.status()` — Setting HTTP Status Codes

> **Cross-reference:** The basics of `res.status()` — setting the status code and chaining with `.send()` or `.json()` — were introduced in **subtopic 07-01 (Express Introduction), Section 5.3**. This section provides a comprehensive status code reference and demonstrates additional patterns.

`res.status()` returns the `res` object itself, which allows **method chaining**:

app.js

```
const express = require('express');
const app = express();

app.use(express.json());

// 201 Created — after successfully creating a resource
app.post('/api/users', function (req, res) {
    const user = { id: 1, name: req.body.name };
    res.status(201).json({ message: 'User created', user: user });
});

// 204 No Content — successful but nothing to return
app.delete('/api/users/:id', function (req, res) {
    console.log('Deleted user ' + req.params.id);
    res.status(204).end();
});

// 400 Bad Request — client sent invalid data
app.post('/api/orders', function (req, res) {
    if (!req.body.product) {
        return res.status(400).json({ error: 'Product field is required' });
    }
    res.status(201).json({ order: { product: req.body.product } });
});

// 404 Not Found
app.get('/api/users/:id', function (req, res) {
    // Simulate: user not in database
    res.status(404).json({ error: 'User not found' });
});

// 500 Internal Server Error
app.get('/api/crash', function (req, res) {
    res.status(500).json({ error: 'Something went wrong on our end' });
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -i -X POST http://localhost:3000/api/users -H "Content-Type: application/json" -d "{\"name\":
\"Alice\"}"
```

Output:

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{"message":"User created","user":{"id":1,"name":"Alice"}}
```

```
curl -i -X DELETE http://localhost:3000/api/users/5
```

Output:

```
HTTP/1.1 204 No Content

(empty body)
```

```
curl -i -X POST http://localhost:3000/api/orders -H "Content-Type: application/json" -d "{}"
```

Output:

```
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{"error":"Product field is required"}
```

Common Status Codes Reference

Code	Name	When to Use
`200`	OK	Default success for GET, PUT, PATCH
`201`	Created	A new resource was created (POST)
`204`	No Content	Success, but nothing to return (DELETE)
`301`	Moved Permanently	Resource has permanently moved to a new URL

Code	Name	When to Use
`302`	Found	Temporary redirect (Express default for `res.redirect`)
`304`	Not Modified	Client cache is still valid (handled automatically by ETags)
`400`	Bad Request	Malformed or invalid input from the client
`401`	Unauthorised	No valid authentication credentials provided
`403`	Forbidden	Authenticated but insufficient permissions
`404`	Not Found	Resource does not exist
`409`	Conflict	Request conflicts with current state (e.g., duplicate entry)
`422`	Unprocessable Entity	Well-formed but semantically invalid data
`429`	Too Many Requests	Rate limit exceeded
`500`	Internal Server Error	Unhandled server-side error
`503`	Service Unavailable	Server temporarily unable to handle requests

Key Insight: `res.status()` does not send the response — it only *sets* the code. You must still call a send method like `.json()`, `.send()`, or `.end()` to actually transmit the response. Calling `res.status(404)` alone does nothing visible to the client.

`res.end()` — Ending Without a Body

`res.end()` sends the response with no body. It is inherited directly from Node's `http.ServerResponse`. The primary use case is responses where a body is not meaningful — such as `204 No Content` or `304 Not Modified`:

app.js

```
const express = require('express');
const app = express();

// HEAD request — client only wants headers, no body
app.head('/api/health', function (req, res) {
    res.status(200).end();
});

// 204 after a delete — confirmed but nothing to return
app.delete('/api/items/:id', function (req, res) {
    console.log('Item ' + req.params.id + ' deleted');
    res.status(204).end();
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -i -X HEAD http://localhost:3000/api/health
```

Output:

```
HTTP/1.1 200 OK
Content-Length: 0

(no body)
```

```
curl -i -X DELETE http://localhost:3000/api/items/7
```

Output:

```
HTTP/1.1 204 No Content

(no body)
```

> **Warning:** Do not call `res.end()` when you intend to send data. It does not set `Content-Type` or perform any encoding. If you need to send a body, always use `res.send()` or `res.json()`.

`res.redirect()` — Redirecting Clients

`res.redirect()` sends a redirect response, instructing the client's browser (or HTTP client) to make a new request to a different URL. By default it sends a `302 Found` status code.

app.js

```
const express = require('express');
const app = express();

// Basic redirect — 302 by default
app.get('/old-page', function (req, res) {
    res.redirect('/new-page');
});

app.get('/new-page', function (req, res) {
    res.send('Welcome to the new page');
});

// Permanent redirect — 301
app.get('/legacy-api', function (req, res) {
    res.redirect(301, '/api/v2');
});

app.get('/api/v2', function (req, res) {
    res.json({ version: 2, status: 'active' });
});

// Redirect to an external URL
app.get('/docs', function (req, res) {
    res.redirect('https://expressjs.com');
```

```
});

// Redirect back to the previous page
app.get('/go-back', function (req, res) {
    res.redirect('back');
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -i http://localhost:3000/old-page
```

Output:

```
HTTP/1.1 302 Found
Location: /new-page
Content-Length: 46

Found. Redirecting to /new-page
```

```
curl -i http://localhost:3000/legacy-api
```

Output:

```
HTTP/1.1 301 Moved Permanently
Location: /api/v2
Content-Length: 50

Moved Permanently. Redirecting to /api/v2
```

The `Location` header tells the client where to go. Browsers follow redirects automatically. API clients like `curl` display the redirect response unless you pass the `-L` flag to follow it:

```
curl -L http://localhost:3000/old-page
```

Output:

```
Welcome to the new page
```

301 vs 302 vs 307 vs 308 — When to Use Which

Code	Meaning	Permanent?	Preserves Method?	Use When
`301`	Moved Permanently	Yes	No — browsers convert POST → GET	Permanent URL change for browser navigation or SEO. **Avoid for APIs.**
`302`	Found (temporary)	No	No — browsers convert POST → GET	Temporary redirect for browser navigation. **Avoid for APIs.**
`307`	Temporary Redirect	No	Yes — method is preserved	Temporary API redirect where the HTTP method must be preserved.
`308`	Permanent Redirect	Yes	Yes — method is preserved	Permanent API endpoint migration (e.g. `/api/users` → `/api/users/v2`).

The Problem with 301 for APIs

301 was designed for web browsers, and it comes with a dangerous historical quirk: when a browser follows a 301, it silently converts any `POST` request into a `GET`. This behaviour was technically incorrect but so common across early browsers that it was formalised in the HTTP spec.

For a web page this is rarely a problem — users click links (`GET` requests). But for a REST API it is catastrophic:

app.js (broken with 301)

```
const express = require('express');
const app = express();
app.use(express.json());

// ❌ Dangerous: 301 causes clients to convert POST → GET on redirect.
// The /api/users/v2 route never receives the request body.
app.post('/api/users/create', function (req, res) {
    res.redirect(301, '/api/users/v2/create');
});

app.post('/api/users/v2/create', function (req, res) {
    console.log(req.body); // {} — body is gone, method changed to GET
    res.status(201).json({ created: true });
});

app.listen(3000);
```

```
curl -i -X POST -H "Content-Type: application/json" -d '{"name":"Alice"}' http://localhost:3000/api/users/create
```

Output:

```
HTTP/1.1 301 Moved Permanently
Location: /api/users/v2/create

Client re-sends as GET /api/users/v2/create — body discarded.
```

Use **308** instead. The 308 status code is the permanent-redirect equivalent of 307: it tells the client that this move is permanent (browsers and search engines can cache and update their records), but the HTTP method and request body must be preserved on the re-request.

app.js (correct with 308)

```
const express = require('express');
const app = express();
app.use(express.json());

// ☑ Correct: 308 preserves the method (POST stays POST) and the request body.
```

```
// Browsers and search engines also treat this as a permanent move,
// so they update their cached URL — same benefit as 301.
app.post('/api/users/create', function (req, res) {
    res.redirect(308, '/api/users/v2/create');
});

app.post('/api/users/v2/create', function (req, res) {
    console.log(req.body); // { name: 'Alice' } — body intact, still POST
    res.status(201).json({ created: true, data: req.body });
});

// Works identically for GET routes — GET stays GET
app.get('/api/users', function (req, res) {
    res.redirect(308, '/api/users/v2');
});

app.get('/api/users/v2', function (req, res) {
    res.json({ version: 2, status: 'active' });
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -i -X POST -H "Content-Type: application/json" -d '{"name":"Alice"}' http://localhost:3000/api/users/create
```

Output:

```
HTTP/1.1 308 Permanent Redirect
Location: /api/users/v2/create
Content-Length: 58

Permanent Redirect. Redirecting to /api/users/v2/create
```

```
curl -L -X POST -H "Content-Type: application/json" -d '{"name":"Alice"}' http://localhost:3000/api/users/create
```

Output:

```
{"created":true,"data":{"name":"Alice"}}
```

Rule of thumb for APIs: Whenever you would reach for `301` (permanent) or `302` (temporary) in an API context, use `308` or `307` respectively. The 3xx codes without method preservation (`301` , `302`) should be reserved for browser-facing page redirects where the GET-conversion behaviour is acceptable.

Note: `res.redirect('back')` uses the `Referer` header to determine the previous page. If no `Referer` header is present (as is often the case with API clients or direct URL entry), Express redirects to `/` as a fallback.

Setting Response Headers

Express sets many headers automatically (`Content-Type` , `Content-Length` , `ETag`), but you often need to set custom headers for caching, security, CORS, or application-specific purposes.

`res.set()` — Setting One or More Headers

app.js

```
const express = require('express');
const app = express();

app.get('/api/data', function (req, res) {
    // Set a single header
    res.set('X-Request-Id', 'req-' + Date.now());

    // Set multiple headers at once
    res.set({
        'Cache-Control': 'no-store',
        'X-Powered-By': 'My Custom Server'
    });

    res.json({ data: 'Some response data' });
});
```

```
app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -i http://localhost:3000/api/data
```

Output:

```
HTTP/1.1 200 OK
X-Request-Id: req-1750070412345
Cache-Control: no-store
X-Powered-By: My Custom Server
Content-Type: application/json; charset=utf-8

{"data":"Some response data"}
```

`res.set()` and `res.header()` are aliases — they do the same thing. Custom headers are conventionally prefixed with `X-`, though this practice is no longer required by the HTTP specification.

`res.type()` — Setting Content-Type Shorthand

`res.type()` is a convenience method that sets the `Content-Type` header. It accepts MIME types, file extensions, or shorthand names:

app.js

```
const express = require('express');
const app = express();

app.get('/plain', function (req, res) {
    res.type('text/plain').send('This is plain text');
});

app.get('/html', function (req, res) {
    res.type('html').send('<p>This is HTML</p>');
});

app.get('/xml', function (req, res) {
    res.type('xml').send('<note><body>XML response</body></note>');
});
```

```
app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -i http://localhost:3000/plain
```

Output:

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8

This is plain text
```

```
curl -i http://localhost:3000/xml
```

Output:

```
HTTP/1.1 200 OK
Content-Type: application/xml; charset=utf-8

<note><body>XML response</body></note>
```

Passing `'html'` expands to `text/html`; passing `'xml'` expands to `application/xml`. This is a simple convenience — it saves typing the full MIME string.

`res.append()` — Adding to Existing Headers

`res.set()` *replaces* a header if it already exists. `res.append()` *adds* a value to an existing header without removing the previous value:

app.js

```
const express = require('express');
const app = express();

app.get('/multi-header', function (req, res) {
    res.append('X-Custom', 'value-one');
    res.append('X-Custom', 'value-two');
    res.json({ message: 'Check the headers' });
});
```

```
app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -i http://localhost:3000/multi-header
```

Output:

```
HTTP/1.1 200 OK
X-Custom: value-one, value-two
Content-Type: application/json; charset=utf-8

{"message":"Check the headers"}
```

Important: Call `res.append()` *before* any send method (`res.send()`, `res.json()`, etc.). Once the response has been sent, headers are already on the wire and cannot be modified.

Serving Files — `res.sendFile()` and `res.download()`

Express provides two methods for sending files to the client. Both read a file from disk and stream it in the response, but they differ in how the browser handles the result.

`res.sendFile()` — Display in the Browser

`res.sendFile()` sends a file and sets the `Content-Type` based on the file extension. The browser will attempt to display the file inline (e.g., render a PDF or show an image).

Create a small test file in the `public` directory:

```
echo "This is a sample report." > public/report.txt
```

`app.js`

```
const express = require('express');
const path = require('path');
const app = express();
```

```
app.get('/view-report', function (req, res) {
    const filePath = path.join(__dirname, 'public', 'report.txt');
    res.sendFile(filePath, function (err) {
        if (err) {
            console.error('Error sending file:', err.message);
            res.status(404).json({ error: 'File not found' });
        }
    });
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -i http://localhost:3000/view-report
```

Output:

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=UTF-8
Content-Length: 27

This is a sample report.
```

`res.sendFile()` requires an **absolute path**. Using `path.join(__dirname, ...)` ensures correctness regardless of the working directory. The optional callback receives an error if the file does not exist or cannot be read.

`res.download()` — Trigger a File Download

`res.download()` is identical to `res.sendFile()` except it sets the `Content-Disposition` header to `attachment`, which prompts the browser to save the file rather than display it. You can optionally specify a different filename for the download:

`app.js`

```
const express = require('express');
const path = require('path');
const app = express();

// Download with the original filename
app.get('/download-report', function (req, res) {
```

```
    const filePath = path.join(__dirname, 'public', 'report.txt');
    res.download(filePath, function (err) {
        if (err) {
            console.error('Download error:', err.message);
            res.status(404).json({ error: 'File not found' });
        }
    });
});

// Download with a custom filename
app.get('/download-renamed', function (req, res) {
    const filePath = path.join(__dirname, 'public', 'report.txt');
    res.download(filePath, 'monthly-report-june.txt', function (err) {
        if (err) {
            console.error('Download error:', err.message);
        }
    });
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -i http://localhost:3000/download-report
```

Output:

```
HTTP/1.1 200 OK
Content-Disposition: attachment; filename="report.txt"
Content-Type: text/plain; charset=UTF-8
Content-Length: 27

This is a sample report.
```

```
curl -i http://localhost:3000/download-renamed
```

Output:

```
HTTP/1.1 200 OK
Content-Disposition: attachment; filename="monthly-report-june.txt"
Content-Type: text/plain; charset=UTF-8
Content-Length: 27

This is a sample report.
```

The `Content-Disposition: attachment` header is the key difference. Without it (`res.sendFile`), the browser displays the file. With it (`res.download`), the browser opens a “Save As” dialog.

Content Negotiation with `res.format()`

Content negotiation is the process where the server examines the client’s `Accept` header and sends back the response in the format the client prefers. `res.format()` makes this straightforward:

app.js

```
const express = require('express');
const app = express();

app.get('/api/user', function (req, res) {
    const user = { id: 1, name: 'Alice', role: 'developer' };

    res.format({
        'application/json': function () {
            res.json(user);
        },
        'text/html': function () {
            res.send('<h1>' + user.name + '</h1><p>Role: ' + user.role + '</p>');
        },
        'text/plain': function () {
            res.type('text/plain').send(user.name + ' (' + user.role + ')');
        },
        default: function () {
            // Fallback when no Accept header matches
            res.status(406).json({ error: 'Not Acceptable' });
        }
    });
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

Test with different `Accept` headers:

```
curl -H "Accept: application/json" http://localhost:3000/api/user
```

Output:

```
{"id":1,"name":"Alice","role":"developer"}
```

```
curl -H "Accept: text/html" http://localhost:3000/api/user
```

Output:

```
<h1>Alice</h1><p>Role: developer</p>
```

```
curl -H "Accept: text/plain" http://localhost:3000/api/user
```

Output:

```
Alice (developer)
```

```
curl -H "Accept: application/xml" http://localhost:3000/api/user
```

Output:

```
{"error":"Not Acceptable"}
```

Express inspects the `Accept` header, matches it against the keys in the object passed to `res.format()`, and calls the corresponding function. The `default` key acts as a catch-all when no match is found. The `406 Not Acceptable` status code tells the client that the server cannot produce a response in the requested format.

> **Key Insight:** Content negotiation is particularly useful when the same API endpoint serves both web browsers (which prefer HTML) and API clients (which prefer JSON). Instead of building separate endpoints, a single route handles both.

Response Lifecycle Rules

Understanding when and how many times you can send a response prevents subtle bugs. Express enforces several rules:

Rule 1: You Can Only Send One Response

Once `res.send()`, `res.json()`, `res.end()`, or `res.redirect()` has been called, the response is finalised and sent to the client. Calling any send method a second time throws an error:

app.js (broken — double send)

```
const express = require('express');
const app = express();

app.get('/double', function (req, res) {
    res.json({ first: true });
    res.json({ second: true }); // Error!
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl http://localhost:3000/double
```

Output:

```
// Client receives the first response:
{"first":true}

// Terminal shows an error:
Error [ERR_HTTP_HEADERS_SENT]: Cannot set headers after they are sent to the client
```

The client receives the first response; the second call throws a server-side error. This commonly happens in code with conditional branches where both branches accidentally send a response:

app.js (common mistake)

```
app.get('/check', function (req, res) {
    if (!req.query.token) {
        res.status(401).json({ error: 'No token' });
        // Missing "return" — execution continues!
    }
    res.json({ data: 'secret' }); // This still runs
});
```

Warning: Always use `return` after sending an early response to prevent further execution. The corrected pattern is:

```
if (!req.query.token) {
    return res.status(401).json({ error: 'No token' });
}
res.json({ data: 'secret' });
```

The `return` stops the function, preventing the second `res.json()` from running.

Rule 2: Set Headers Before Sending

Headers must be set before any send method is called. Once the response body begins transmitting, headers are already on the wire:

app.js (correct order)

```
app.get('/ordered', function (req, res) {
    // 1. Set headers first
    res.set('X-Request-Id', 'abc-123');
    res.type('json');

    // 2. Set the status code
    res.status(200);

    // 3. Send the body last
    res.json({ message: 'Headers were set before sending' });
});
```

Rule 3: Use `res.headersSent` to Check

If you are unsure whether a response has already been sent (e.g., inside error-handling middleware), check `res.headersSent`:

`app.js (safe error handler)`

```
app.use(function (err, req, res, next) {
    if (res.headersSent) {
        // Response already sent — delegate to Express default handler
        return next(err);
    }
    res.status(500).json({ error: err.message });
});
```

Output:

```
// No output to show — this is a defensive pattern inside error-handling middleware.
```

`res.headersSent` is a boolean that becomes `true` as soon as any response data has been written. This check prevents the "headers already sent" error in edge cases where an error occurs after a partial response.

Choosing the Right Method — Quick Decision Guide

Scenario	Method	Why
Return a JSON object or array	`res.json()`	Clearest intent; handles `null`, numbers, booleans correctly
Return an HTML string	`res.send()`	Auto-sets `text/html`; works for simple HTML responses
Return plain text	`res.type('text').send()`	Override the default `text/html` content type

Scenario	Method	Why
Return no body (204, 304)	`res.status(204).end()`	No body to encode; `end()` closes the response
Send the client to another URL	`res.redirect()`	Sets `Location` header and appropriate 3xx status
Display a file (image, PDF)	`res.sendFile()`	Streams the file; browser displays it inline
Trigger a file download	`res.download()`	Adds `Content-Disposition: attachment` ; browser saves the file
Respond based on client's Accept header	`res.format()`	Content negotiation — serve JSON, HTML, or text from one route

---

Summary

Concept	Key Point
`res.send()`	General-purpose sender; auto-detects Content-Type from data type (string → HTML, object → JSON, Buffer → octet-stream)
`res.json()`	Purpose-built for JSON; handles `null` , booleans, numbers; signals intent clearly
`res.end()`	Ends response with no body; use for `204` , `304` , and HEAD requests
`res.status()`	Sets the HTTP status code; chainable; does *not* send the response on its own

Concept	Key Point
`res.redirect()`	Sends 302 (default) or 301 redirect with a `Location` header
`res.set()` / `res.header()`	Sets one or more response headers; must be called before sending
`res.type()`	Shorthand for setting `Content-Type`; accepts extensions (`'json'`, `'html'`)
`res.append()`	Adds a value to an existing header instead of replacing it
`res.sendFile()`	Streams a file for inline display; requires an absolute path
`res.download()`	Streams a file as a download attachment; sets `Content-Disposition`
`res.format()`	Content negotiation: matches the client's `Accept` header to response handlers
`res.headersSent`	Boolean; `true` after a response has started sending; use in error handlers to prevent double-send
Single-response rule	Only one send method per request; use `return` after early responses to prevent double-send errors
`json spaces` setting	`app.set('json spaces', 2)` enables pretty-printed JSON; disable in production

Interview Questions

1. What is the difference between `res.send()` and `res.json()`? When would you use each?
Expected answer: Both can send JSON. `res.send()` auto-detects the content type: strings become `text/html`, objects become `application/json`. `res.json()` always sends JSON and correctly handles edge cases like `null`, `true`, and numbers. Use `res.json()` for API responses (clear intent) and `res.send()` for HTML strings or other mixed content.

2. What happens if you call `res.json()` twice in the same route handler?
Expected answer: The first call sends the response successfully. The second call throws an `ERR_HTTP_HEADERS_SENT` error because the response (including headers) has already been sent. You can only send one response per request. Use `return` before early responses to prevent this.

3. Does `res.status(404)` send a response to the client?
Expected answer: No. `res.status()` only sets the status code internally and returns the `res` object for chaining. You must still call a send method like `.json()` or `.end()` to actually transmit the response.

4. What is the default HTTP status code for `res.redirect()`, and how do you change it?
Expected answer: The default is `302 Found` (temporary redirect). Pass the desired code as the first argument: `res.redirect(301, '/new-url')` for a permanent redirect.

5. Explain the difference between `res.sendFile()` and `res.download()`.
Expected answer: Both stream a file from disk to the client. `res.sendFile()` sends the file for inline display — the browser renders it (e.g., shows a PDF). `res.download()` adds a `Content-Disposition: attachment` header, which prompts the browser to save the file instead of displaying it.

6. How does `res.format()` decide which response to send?
Expected answer: It reads the request's `Accept` header and matches it against the MIME types provided in the format object. It calls the function for the best match. If no match is found and a `default` handler exists, that runs; otherwise Express sends a `406 Not Acceptable` response.

7. Why must response headers be set before calling `res.send()` or `res.json()`?
Expected answer: HTTP responses transmit headers before the body. Once a send method is called, Express writes the headers to the network stream and begins sending the body. After that, modifying headers is impossible — they are already on the wire. This is why `res.set()` must come before `res.send()`.

8. How would you check in error-handling middleware whether a response has already been sent?
Expected answer: Check `res.headersSent`. If it is `true`, the response is already in progress and you should call `next(err)` to delegate to Express's default handler rather than trying to send a second response.

Practical Assignment: Build a Multi-Format File Server API

Objective: Create an Express application that demonstrates every response method covered in this subtopic: `res.send()`, `res.json()`, `res.status()`, `res.redirect()`, `res.set()`, `res.sendFile()`, `res.download()`, and `res.format()`.

1. Create a new project:
   ```
   mkdir file-server-api && cd file-server-api
   npm init -y
   npm install express
   mkdir public
   ```
2. Create a test file:
   ```
   echo "Quarterly sales report: Q2 2025 revenue was 1.4M." > public/report.txt
   ```
3. Create `app.js` with the following routes:
   1. `GET /` — returns an HTML welcome page using `res.send()`. Set a custom header `X-App-Version: 1.0.0`.
   2. `GET /api/status` — returns JSON `{ status: "healthy", uptime: process.uptime() }` using `res.json()`. Set `Cache-Control: no-store`.

3. `GET /api/report` — implements content negotiation with `res.format()`:
   - `application/json` → respond with `{ title: "Q2 Report", revenue: 1400000 }`
   - `text/html` → respond with an HTML snippet showing the report title in an `<h1>`
   - `text/plain` → respond with a plain-text summary
   - `default` → respond with `406 Not Acceptable`
4. `GET /api/report/view` — serves `public/report.txt` inline using `res.sendFile()`.
5. `GET /api/report/download` — serves `public/report.txt` as a download named `q2-report-2025.txt` using `res.download()`.
6. `GET /old-report` — permanently redirects (301) to `/api/report`.
7. `DELETE /api/report` — returns `204 No Content` using `res.status(204).end()`.

4. Add a 404 catch-all and an error handler at the end (as covered in Section 7.03).
5. Test every route and verify:

Expected output:

```
curl -i http://localhost:3000/
```

Output:

```
HTTP/1.1 200 OK
X-App-Version: 1.0.0
Content-Type: text/html; charset=utf-8

<h1>Welcome to the File Server API</h1>
```

```
curl http://localhost:3000/api/status
```

Output:

```
{"status":"healthy","uptime":12.345}
```

```
curl -H "Accept: application/json" http://localhost:3000/api/report
```

Output:

```
{"title":"Q2 Report","revenue":1400000}
```

```
curl -H "Accept: text/plain" http://localhost:3000/api/report
```

Output:

```
Q2 Report: Revenue was 1.4M
```

```
curl -H "Accept: application/xml" http://localhost:3000/api/report
```

Output:

```
{"error":"Not Acceptable"}
```

```
curl -i http://localhost:3000/api/report/view
```

Output:

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=UTF-8

Quarterly sales report: Q2 2025 revenue was 1.4M.
```

```
curl -i http://localhost:3000/api/report/download
```

Output:

```
HTTP/1.1 200 OK
Content-Disposition: attachment; filename="q2-report-2025.txt"
Content-Type: text/plain; charset=UTF-8

Quarterly sales report: Q2 2025 revenue was 1.4M.
```

```
curl -i -L http://localhost:3000/old-report
```

Output:

```
HTTP/1.1 301 Moved Permanently
Location: /api/report
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"title":"Q2 Report","revenue":1400000}
```

```
curl -i -X DELETE http://localhost:3000/api/report
```

Output:

```
HTTP/1.1 204 No Content

(no body)
```

Bonus Challenge: Add a `GET /api/report/stream` route that reads `report.txt` using `fs.createReadStream()` (Section 5) and pipes it directly to the response with `res.type('text/plain')`. This demonstrates the streaming pattern that `res.sendFile()` uses internally.